Basic Information

- Name: Richard (Yunchao) He
- Student ID: 2301773
- Report document: https://docs.google.com/document/d/1-ZDW3gfpp8INP-OkVTRbdMl9UA9bLznI8Lt3N-nxffo/edit?tab=t.0#

Problem 1 - Part (e)

Possibility of Further Model Size Reduction
Can you **further reduce the model size** beyond the smallest model obtained in parts **(b)**, **(c)**, or **(d)**, **without sacrificing significant classification performance**?

Your task is to:

1. **Analyze and compare** the results from previous parts: Which model had the smallest size? Which performed best?

The results from the baseline model and the three compression methods are summarized below.

Model	Parameter/Compression Method	TFLite Size	Test Accuracy
Baseline model	3,075 parameters, Float32	14.07 KB	96.30%

Dynamic-range quantization	Quantized weights with Float32 input/output	8.17 KB	96.30%
Float16 quantization	Float16 weights with Float32 input/output	8.95 KB	96.30%
Full-integer quantization	INT8 weights, activations, input, and output	**5.74 KB**	96.30%
Pruned model	68.95% kernel sparsity, Float32	14.14 KB	96.30%
Knowledge-distilled student	1,027 parameters, Float32	6.12 KB	96.30%

Among the models evaluated so far, **the full-integer quantized baseline model had the smallest TFLite file size at 5.74 KB**. Compared with the 14.07 KB Float32 baseline model, this represents a size reduction of approximately 59.2%. The knowledge-distilled student was the second-smallest model at 6.12 KB, representing a reduction of approximately 56.5% relative to the baseline.

In terms of overall classification performance, all models achieved the same test accuracy of 96.30%, correctly classifying 52 of the 54 test samples. Therefore, there was no single model that performed better based on test accuracy alone. The models did, however, make slightly different individual classification errors.

The baseline and all three quantized models produced the following confusion matrix:

```
None
      [[18  0  0]
```

```
 [ 1 19  1]
 [ 0  0 15]]
```

```
None
[[18  0  0]
 [ 1 20  0]
 [ 0  1 14]]
```

Both groups made two incorrect predictions, but the errors occurred in different classes. These differences were too small to establish a significant performance advantage for one model, especially because the test set contained only 54 samples.

The pruning experiment achieved an overall kernel sparsity of 68.95%, meaning that approximately 69% of the Dense-layer kernel weights were exactly zero. Nevertheless, the raw TFLite file size remained approximately unchanged, increasing slightly from 14.07 KB to 14.14 KB. This occurred because a standard dense TFLite representation still stores every Float32 weight explicitly. A zero-valued Float32 weight occupies the same four bytes as a nonzero Float32 weight. Consequently, pruning created sparsity but did not translate that sparsity into a smaller uncompressed TFLite file. A sparsity-aware storage format, runtime, or general-purpose compression method would be required to obtain a file-size benefit from the zero weights.

Knowledge distillation reduced the number of model parameters from 3,075 in the teacher to 1,027 in the student, a reduction of approximately 66.6%. The Student architecture was reduced from 13−64−32−3 to 13−32−16−3. Despite this substantial structural reduction, the Student retained the teacher's test accuracy of 96.30%. Its Float32 TFLite file was 6.12 KB, slightly larger than the 5.74 KB INT8 baseline because its smaller set of parameters was still stored in Float32 format.

Considering both size and performance, the full-integer quantized baseline was the best current deployment candidate. It had the smallest file size, preserved the baseline accuracy, and used INT8 input, output, weights, and activations, making it particularly appropriate for resource-constrained TinyML hardware.

2. **Propose a strategy** that combines or enhances techniques learned so far.

   I propose **applying full-integer quantization to the knowledge-distilled Student model**. This combines two complementary compression methods: knowledge distillation reduces the number of parameters, while INT8 quantization reduces the storage required for each parameter and activation.

   The baseline Teacher contains 3,075 parameters, whereas the Student contains only 1,027 parameters. The current Student model is stored in Float32 format and has a TFLite size of 6.12 KB. Converting it to full INT8 should reduce its size further because eligible values will generally require one byte instead of four bytes. The resulting model is expected to be smaller than the current smallest model—the 5.74 KB INT8 baseline—while retaining accuracy close to 96.30%.

   This strategy is more suitable than combining pruning with quantization in this experiment because the standard dense TFLite format still stores zero-valued pruned weights explicitly. In contrast, knowledge distillation removes parameters from the architecture itself, and quantization then compresses the remaining parameters. Therefore, combining knowledge distillation with full-integer quantization should provide both structural and numerical compression.

3. **Implement** your proposed solution.

   See the implementation code at .ipynb file

4. **Evaluate** the resulting model using both:
   - TFLite model size (in KB)
   - Classification performance (accuracy and report)

   See the evaluation code at .ipynb file.

Combining knowledge distillation with full-integer quantization reduced the TFLite model size to **3.66 KB**, which was 36.2% smaller than the previous smallest model and approximately 74.0% smaller than the original Float32 baseline. **The model retained a test accuracy of 96.30%**, with no change in its classification report or confusion matrix compared with the Float32 Student.

See the detailed report from the screenshot of evaluation code execution at .ipynb:

```
KD + INT8 TFLite model size: 3.66 KB

Input tensor:
shape = [ 1 13] dtype = <class 'numpy.int8'> quantization = (0.030695566907525063, -6)
Output tensor:
shape = [1 3] dtype = <class 'numpy.int8'> quantization = (0.00390625, -128)

KD + INT8 test accuracy: 0.9630 (96.30%)

KD + INT8 classification report:
              precision    recall  f1-score   support

           0     0.9474    1.0000    0.9730        18
           1     0.9524    0.9524    0.9524        21
           2     1.0000    0.9333    0.9655        15

    accuracy                         0.9630        54
   macro avg     0.9666    0.9619    0.9636        54
weighted avg     0.9639    0.9630    0.9629        54

KD + INT8 confusion matrix:
[[18  0  0]
 [ 1 20  0]
 [ 0  1 14]]
```

5. **Justify your results:**
    - If further size reduction is **not** possible without major loss of accuracy, explain why.

Not applicable. Further size reduction was successfully achieved without any loss in test accuracy.

- If you succeed in reducing the size **further**, highlight what change made the biggest difference.

  The proposed method successfully reduced the TFLite model size to 3.66 KB while preserving the test accuracy of 96.30%. This model was 36.2% smaller than the previous smallest model, the 5.74 KB INT8 baseline, and approximately 74.0% smaller than the original 14.07 KB Float32 baseline.

  The most important change was applying full-integer quantization to the already reduced knowledge-distilled Student rather than to the larger baseline model. Knowledge distillation first reduced the parameter count from 3,075 to 1,027, and INT8 quantization then reduced the storage precision of the remaining weights and activations. Quantizing the Student reduced its size from 6.12 KB to 3.66 KB, a further reduction of approximately 40.2%.

  Therefore, the improvement resulted from combining structural compression with numerical compression. Knowledge distillation removed parameters from the network, while INT8 quantization reduced the storage cost of the remaining parameters. Neither technique alone produced a model as small as the combined approach, and the quantized Student retained the same classification report and confusion matrix as the Float32 Student.

Problem 2: Exploring Edge Impulse (20 points)

1. [Dis] (5 pts) What types of “Learning Blocks” are available under the free-tier Edge Impulse account (e.g., supervised learning—classification and regression, unsupervised learning—clustering, self-supervised learning, etc.)?
For each type, provide at least one example model that can be implemented using the free tier.

Edge Impulse’s free Developer plan supports several built-in Learning Blocks covering supervised learning, unsupervised anomaly detection, and transfer-learning-based tasks. The main categories and example models are summarized below.

Learning type	Available Learning Blocks	Purpose	Example model

Supervised classification	Classification (Keras)	Learns discrete class labels from labeled feature data.	An accelerometer-based neural network that classifies motion as idle, walking, or running.
Supervised regression	Regression (Keras)	Predicts a continuous numerical value rather than a class label.	A vibration-based model that estimates motor speed or remaining machine life.
Supervised image/audio classification through transfer learning	Image Classification and Keyword Spotting	Fine-tunes a pretrained feature extractor for a new labeled image or audio dataset.	A MobileNet-based mask/no-mask image classifier or a keyword-spotting model that recognizes “yes” and “no.”
Supervised object detection	FOMO and MobileNetV2 SSD FPN	Identifies objects and their locations within an image.	A FOMO model that detects and counts people or objects on a microcontroller.
Unsupervised anomaly detection	K-means and Gaussian Mixture Model (GMM) anomaly detection	Models normal feature patterns and assigns a high anomaly score to unfamiliar observations.	A K-means model that detects abnormal machine vibration or an unknown gesture.
Visual anomaly detection	FOMO-AD	Locates unusual regions or defects in images by learning the appearance of normal samples.	A visual inspection model that detects scratches or defects on manufactured products.

The official Edge Impulse documentation lists Classification, Regression, K-means anomaly detection, GMM anomaly detection, FOMO-AD, image transfer learning, keyword-spotting transfer learning, FOMO object detection, and MobileNetV2 SSD FPN object detection as predefined Learning Blocks. Edge Impulse Learning Blocks documentation

The K-means and GMM blocks are considered unsupervised anomaly-detection methods because they learn the distribution or clusters of normal feature data rather than learning explicit anomaly classes. K-means represents normal data using cluster centroids, while GMM models it as a mixture of Gaussian probability distributions. K-means documentation, GMM documentation

Edge Impulse does not currently provide a separate predefined Learning Block explicitly labeled “self-supervised learning” in the standard Studio list. However, custom Learning Blocks are available to all users, including users of the free Developer plan. Therefore, an advanced user could implement a self-supervised model, such as an autoencoder or a contrastive representation-learning model, through a custom Keras or PyTorch training block. This should be distinguished from the built-in Learning Blocks listed above. Custom Learning Blocks documentation

Classical ML is listed in the general Learning Blocks documentation, but its built-in Studio block is currently marked as Enterprise-only. Therefore, built-in logistic regression, SVM, random forest, XGBoost, and LightGBM should not be counted as free-tier predefined Learning Blocks. Edge Impulse Classical ML documentation

2. [Dis] (5 pts) List all the types of layers that can be used under the “Classifier” section in the free-tier Edge Impulse. Briefly explain the purpose of each layer identified.

In the visual Neural Network Architecture editor under the Edge Impulse Classifier, the following layer types can be used:

Layer type	Purpose
Dense (fully connected)	Connects every input from the previous layer to every neuron in the current layer. Dense layers learn combinations of extracted features and are commonly used for final classification decisions.
Dropout	Randomly disables a specified fraction of neurons during training. It is a regularization method that reduces overfitting and encourages the model to learn more robust features.

Reshape	Changes the dimensions of a tensor without changing its underlying values. It is used when the next layer requires a specific input shape, such as converting a flat feature vector into a sequence or 2D feature map.
Flatten	Converts multidimensional feature maps into a one-dimensional vector, typically before passing them to a Dense layer.
1D Convolution	Applies learnable filters along one dimension to detect local patterns in sequential data, such as time-series sensor data or audio features.
1D Max Pooling	Reduces the length of a 1D feature map by keeping the maximum value from each local window. It preserves the strongest detected features while reducing computation.
1D Average Pooling	Reduces the length of a 1D feature map by calculating the average value within each local window. It summarizes local features more smoothly than max pooling.
2D Convolution	Applies two-dimensional filters to data such as images or spectrograms. It learns spatial features including edges, shapes, textures, and local time-frequency patterns.
2D Max Pooling	Reduces the height and width of 2D feature maps by retaining the maximum value from each local region. It lowers memory and computation while preserving strong feature responses.
2D Average Pooling	Reduces the height and width of 2D feature maps by averaging each local region. It provides a smoother spatial summary of the features.

The model also contains an **input layer**, whose shape is determined by the features produced by the preceding processing block. The input layer receives data but does not perform a learned transformation. For classification, Edge Impulse automatically uses an **output layer** whose size matches the number of classes and whose softmax activation produces a probability distribution across those classes.

The official Edge Impulse documentation groups the manually configurable layers as Dense, Reshape, Flatten, Dropout, 1D convolution, 1D pooling, 2D convolution, and 2D pooling. Max and average pooling are the two supported pooling operations, so they are listed separately above as selectable layer variants. Edge Impulse neural-network layer documentation

These are the layers available through the visual Classifier editor. Edge Impulse also provides a Keras Expert Mode, which permits more advanced architectures through the Keras API, but those additional Keras layers are not part of the standard visual Classifier layer list. Edge Impulse Learning Blocks documentation

3. [Dis] (5 pts) What types of model compression options are available under the “Deployment” section in the free-tier Edge Impulse? List all available methods.

The free-tier Edge Impulse Deployment section provides two main methods for reducing a model’s deployment footprint:

1. **INT8 Quantization**
   The model can be deployed as a quantized INT8 model instead of an unoptimized Float32 model. Quantization converts eligible model weights and activations from 32-bit floating-point values to 8-bit integers. This generally reduces model size, RAM and flash usage, and inference latency. However, because INT8 represents values with lower numerical precision, it can sometimes cause a small reduction in model accuracy.
2. **EON Compiler**
   The standard EON Compiler converts the model into hardware-optimized C++ code for edge deployment. It performs model-graph and memory optimizations that can reduce RAM and flash consumption and improve inference speed, generally without changing the model’s classification accuracy. Edge Impulse reports that the EON Compiler can reduce RAM usage by approximately 25–65% and flash usage by approximately 10–35%, depending on the model architecture and supported operators.

The Deployment page also allows the user to select the following reference options:

- **Unoptimized Float32 model:** Retains the original 32-bit floating-point representation. It generally provides the highest numerical precision but has a larger memory and storage footprint.
- **LiteRT/TFLite runtime:** Uses the standard TensorFlow Lite–based inference runtime rather than the EON Compiler. This is the baseline runtime option and is not itself an additional compression method.

The free-tier options can therefore be combined in several ways:

Model precision	Inference engine
Float32	LiteRT/TFLite
INT8	LiteRT/TFLite
Float32	EON Compiler
INT8	EON Compiler

For the greatest reduction in resource usage, a user can combine **INT8 quantization with the EON Compiler**.

Edge Impulse also provides an **EON Compiler (RAM optimized)** mode, which can reduce peak RAM usage further by calculating some intermediate values only when needed. However, the official documentation marks this mode as Enterprise-only, so it should not be included among the free-tier compression options.

Therefore, the model-compression and optimization methods available in the free tier are:

- **Full INT8 quantization**
- **Standard EON Compiler**
- **INT8 quantization combined with the standard EON Compiler**

The Float32 and LiteRT/TFLite choices are available deployment configurations, but they serve as uncompressed baseline options rather than additional compression techniques.

Sources: Edge Impulse Deployment documentation, EON Compiler documentation

4. [Dis] (5 pts) What input modalities are supported for synthetic data generation in the free-tier version of Edge Impulse? For each modality, list the synthetic data generation methods/algorithms provided.

Briefly explain why synthetic data is important in machine learning applications.

The free-tier Edge Impulse Synthetic Data feature currently supports two built-in input modalities: **images** and **speech/audio**.

Input modality	Synthetic-data method or algorithm	Description
Image	**DALL-E Image Generation Block**	Uses OpenAI's DALL-E model to generate labeled images from natural-language prompts. For example, it can generate images of factory workers wearing hard hats or aerial views of streets.
Image	**FLUX Pro Image Generation Block**	Uses the FLUX Pro text-to-image model through Replicate to generate high-quality images from textual descriptions. It can create controlled variations in objects, backgrounds, viewpoints, and visual conditions.
Speech/audio	**Whisper Keyword Spotting Generation Block**	Generates speech samples from a supplied keyword or phrase for keyword-spotting and voice-command datasets. It is intended for applications such as voice recognition and command-and-control systems.

The DALL-E block requires an OpenAI API key, while FLUX Pro requires a Replicate account and API key. The Whisper keyword-spotting block also requires an OpenAI API key. Therefore, these blocks are accessible through the free Edge Impulse Developer plan, but use of the external model providers may still be subject to their own usage limits or charges.

Edge Impulse also lists a **Time-Series Data Augmentation Block**, which produces new sensor time-series samples from existing data. However, the official documentation marks this block as Enterprise-only, so time-series generation should not be included as a free-tier built-in modality. Custom Synthetic Data Blocks are also Enterprise-only.

Synthetic data is important in machine learning for several reasons:

- **Lower collection cost and time:** Large datasets can be generated more quickly and cheaply than collecting every sample from physical sensors or real environments.
- **Greater diversity:** Synthetic generation can introduce different backgrounds, lighting conditions, speakers, viewpoints, and other controlled variations.
- **Rare and dangerous scenarios:** It can represent events that are uncommon, expensive, unsafe, or difficult to capture in the real world.
- **Class balancing:** Additional samples can be generated for underrepresented classes, reducing dataset imbalance.
- **Privacy:** Synthetic samples can supplement or replace sensitive real-world data, such as personal images or voice recordings.
- **Improved robustness:** When realistic synthetic variations supplement real data, the model can generalize better to previously unseen conditions.

However, synthetic data should normally supplement rather than completely replace real data. If generated samples fail to represent the actual deployment environment, the model may learn artificial patterns and perform poorly on real inputs. Final validation should therefore use representative real-world test data.

Source: Edge Impulse Synthetic Data documentation, Synthetic data concepts and benefits, Custom Synthetic Data Blocks